ASSIGNMENT 1

DESIGN, IMPLEMENT AND VALIDATE THE FARE LOGIC OF A CITY METRO SMART-CARD SYSTEM

Name: Nokeshwaran S

Register Number: 192511007

Course Code: DSA0101

Course: Object Oriented Programming Using C++

Submission Date: 21/10/2026

GitHub Repository Link: https://github.com/Nokeshwaran27/DSA0101-Object-Oriented-Programming-using-C-.git

Contents

- Scenario and assumptions
- Task 1 – Objects and OOP
- Task 2 – Tokens and constants
- Task 3 – Data types
- Task 4 – Implicit conversions
- Task 5 – Operators and precedence
- Task 6 – Control structures
- Task 7 – Procedural and OOP versions
- Task 8 – Safe input and output
- Task 9 – Implementation and testing
- Task 10 – Expected versus actual execution
- Task 11 – Final recommendation
- Task 12 – Design challenge
- Conclusion and submission checklist

Scenario and Assumptions

The metro has Inner, Middle and Outer zones with base fares of Rs 10, Rs 20 and Rs 30. Peak hours are 08:00 to before 10:00 and 17:00 to before 20:00. A 20% surcharge applies during peak hours. A journey is allowed only if at least Rs 50 remains after deduction. Student and senior-citizen concessions are mentioned, but their percentages are not specified; this report recommends a configurable concession rule without inventing a rate.

Money is stored as integer paise: Rs 10 equals 1000 paise. The intervals are half-open: 08:00 and 17:00 are peak, while 10:00 and 20:00 are off-peak.

Task 1 – Understand the Problem and Identify the Objects

Object	Data	Behaviour and justification
SmartCard	Card ID and balance in paise	canTravel, deduct and recharge protect balance rules.
Passenger	ID and passenger category	Identifies student or senior concession eligibility.
Journey	Zone, hour and date	Represents a trip and supplies fare inputs.
Zone	Inner, Middle or Outer	A distinct type selects the correct fare.
FareTable	Base fares and surcharge rules	Centralizes pricing so revisions happen in one place.

OOP principles

- Encapsulation: SmartCard balance is private and changed only through checked methods.
- Abstraction: the gate calls canTravel(fare) without knowing internal validation details.
- Inheritance: StudentPassenger and SeniorPassenger could inherit common data and behaviour from Passenger.
- Polymorphism: a virtual concessionRate method could return a different concession for each passenger category.

Rules and procedural comparison

- After a successful journey, balance must be at least 5000 paise.
- A failed journey must not deduct money; a fare must not be negative.
- Only valid zones and hours are accepted; money calculations use integer paise.

A procedural design uses separate variables and free functions such as calculateFare and canTravel. An OOP design groups related state and behaviour in classes. Both take O(1) time per journey and O(1) memory per card. OOP makes fare changes easier when rules are centralized instead of duplicated.

Task 2 – Tokens, Keywords, Identifiers and Constants

Token category	Example	Meaning
Keyword	class, switch, return	Reserved C++ word.
Identifier	SmartCard, balance	Programmer-defined name.
Constant/literal	5000, 20, true	Fixed value.
Operator	+, %, >=, &&	Calculation or comparison.
Punctuator	braces, parentheses, semicolon	Program structure.

```
constexpr long long MIN_BALANCE_PAISE = 5000;

constexpr int PEAK_SURCHARGE_PERCENT = 20;
```

```
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
```

#define is preprocessor text substitution and has limited type checking and scope. const prevents modification after initialization. constexpr supports compile-time evaluation and suits fixed fares. enum class Zone { Inner, Middle, Outer } is safer than an ordinary enum because names are scoped and do not implicitly convert to int. Use meaningful identifiers, avoid keywords and spaces, and follow consistent naming conventions.

Macros and constexpr constants usually need no runtime storage when optimized. constexpr and enum class provide better type checking. Adding a fourth zone is clearer with one centralized fare table than with scattered macros.

Task 3 – Data Types and Type Compatibility

Value	Suggested type	Reason
Fare and balance	long long paise	Exact integer money and large range.
Hour	int	Small integer from 0 to 23.
Zone	enum class Zone	Type-safe zone value.
Peak indicator	bool	True or false.
Journey count	long long	Large totals.
Percentage	int	Whole percentage such as 20.
Names	std::string	Stores text labels.
Temporary value	auto	Useful when the initializer makes the type clear.

Common 64-bit g++ sizes are bool 1 byte, char 1, int 4, unsigned 4, long long 8, float 4 and double 8. Sizes depend on the implementation; verify with sizeof on the actual compiler. The climits header provides limits such as INT_MAX and LLONG_MAX.

```
std::cout << sizeof(int) << sizeof(long long);
```

Money should be stored in paise because decimal fractions are not always exactly represented by binary floating point. A 20% surcharge on 3000 paise is 3000 multiplied by 20 and divided by 100, which gives exactly 600 paise. In mixed expressions, bool and char usually undergo integral promotion; an integer is converted to double when combined with double arithmetic; integer division truncates toward zero. Signed/unsigned comparison can convert a negative signed value to a large unsigned value.

At Rs 30 per journey, 3,000,000 journeys total Rs 90,000,000 or 9,000,000,000 paise. This exceeds the usual 32-bit signed int maximum, so long long is required for the paise total. Integer and double addition are O(1); each long long or double commonly uses 8 bytes. Integer paise is exact within range; double can accumulate rounding error.

Task 4 – Control Implicit Conversions

- Integer division: 7 divided by 2 is 3 before assignment to double. An explicit cast before division produces 3.5.
- Character promotion: ch + 1 converts char to int for arithmetic.
- Integer-to-bool: assigning 5 to bool converts the nonzero value to true; default output is 1, while boolalpha prints true.
- Signed-to-unsigned: comparing int s=-1 with unsigned u=1 commonly converts -1 to a large unsigned value, so s<u is false.
- Narrowing: int x{3.7} is rejected by brace initialization. Validate before an intentional cast.

Predicted output

```
3 3.5

C 68

1 true

s >= u
```

Explanation: integer division makes c equal 3. The explicit cast makes d equal 3.5. Character A plus 2 prints C; adding 1 to ch is an integer expression and prints 68. The value 5 converts to true, shown first as 1 and then as true after boolalpha. The signed/unsigned comparison makes s<u false on a typical implementation. Explicit casts and brace initialization improve clarity and can make narrowing errors compile-time errors; runtime and memory complexity do not change.

Task 5 – Operators and Precedence

```
bool peak = (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);

if (peak) fare += baseFare * PEAK_SURCHARGE_PERCENT / 100;

if (concession) fare = fare * (100 - concessionPercent) / 100;
```

Multiplication and division occur before addition. Comparisons are evaluated before logical AND and OR. Parentheses make the two peak intervals clear; the unparenthesized expression is logically equivalent because && has higher precedence than ||. The conditional operator ?: selects between two values. x++ yields the old value and then increments; ++x increments first. Avoid i=i++; use a separate increment statement. Integer division truncates toward zero: 10/3 is 3 and -10/3 is -3; remainders are 1 and -1 respectively.

Predicted output

```
11 16

9 6 5

6 4

3 1 -3 -1

18
```

First line: 5+3*2=11 and (5+3)*2=16. In z=x++ + ++y, x++ contributes 5 and increments x to 6; ++y increments y to 4 and contributes 4, so z=9. Next, max(x,y)=6 and (x%4)*2=4. Division and remainder output 3, 1, -3, -1. The loop adds 0+3+6+9=18.

A bool peak[24] lookup table and a combined logical expression both take O(1) time. The table uses about 24 bytes if bool is one byte and may reduce comparisons, but must be updated when policy changes.

Task 6 – Design the Control Structures

Use switch to select a zone fare, if-else for validation and surcharge, and the conditional operator only for short readable choices. Use a for loop when journey count is known, a range-based for loop for a collection of trips, and while when reading until a sentinel or end-of-file. continue can skip an invalid journey; break can stop after a fatal error. If no switch case matches and there is no default, no case body runs.

Pseudocode

```
START

Read zone, hour and balance.

If zone is invalid or hour is outside 0..23, reject.

Otherwise select base fare by zone.

Peak = (hour >= 8 AND hour < 10) OR (hour >= 17 AND hour < 20).

If peak, fare = base + base*20/100; otherwise fare = base.

If balance - fare >= 5000, deduct and accept; otherwise deny without change.

STOP
```

Completed TODO logic

```
enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};

long long fare(Zone z, bool peak) {

  long long base = 0;

  switch (z) { case Zone::Inner: base=BASE_PAISE[0]; break;

    case Zone::Middle: base=BASE_PAISE[1]; break;

    case Zone::Outer: base=BASE_PAISE[2]; break; }

  if (peak) base += base*20/100;

  return base;

}

bool canTravel(long long balance, long long f) { return f>=0 && balance-f>=5000; }
```

The assignment program expected output is: Fare Rs 10.00 balance Rs 90; Fare Rs 36.00 balance Rs 54; Denied (low balance) balance Rs 54. At 08:00 and 17:00 the trip is peak; at 10:00 and 20:00 it is off-peak. A switch and array lookup are O(1); a fare array uses O(z) space and makes extension straightforward when indexes are validated.

Task 7 – From Procedural to Object-Oriented

Procedural version

```
bool canTravel(long long balance,long long fare) {

  return fare>=0 && balance-fare>=5000;

}

void deduct(long long& balance,long long fare) {

  if (canTravel(balance,fare)) balance-=fare;

}
```

Object-oriented version

```
class SmartCard {

 private: long long balancePaise;

 static constexpr long long MIN_BALANCE_PAISE=5000;

 public:

 explicit SmartCard(long long initial):balancePaise(initial<0?0:initial) {}

 long long balance() const { return balancePaise; }

 bool canTravel(long long f) const { return f>=0 && balancePaise-f>=MIN_BALANCE_PAISE; }

 bool deduct(long long f) { if(!canTravel(f)) return false; balancePaise-=f; return true; }

};
```

The procedural version can work, but code with access to the balance can modify it incorrectly. SmartCard keeps the balance private and deducts only after validation. Both versions take O(1) time and O(1) space per card. If balance were public, code could bypass the Rs 50 minimum or set a negative balance. A centralized minimum-balance constant makes policy changes easier.

Task 8 – Read and Print Data Safely

```
if (!(std::cin >> hour)) {

  std::cin.clear();

  std::cin.ignore(std::numeric_limits<std::streamsize>::max(), newline);

}
```

A failed cin extraction detects letters entered where a number is expected. cin.clear resets the stream error state and ignore discards the invalid line. The actual C++ character argument for newline should be written as a newline character literal. Keep money in paise and format only when printing. setw(2) and setfill(0) can print two paise digits; fixed and setprecision(2) can format a display-only rupee value.

An alternative is getline followed by stoi in try/catch. Both approaches need O(L) time to process a line of length L and O(L) buffer space. Direct extraction is concise; getline plus conversion provides more control over the whole line and trailing characters.

Task 9 – Implement and Test the Fare Logic

Test	Input	Expected result
1	Inner, 12:00, balance Rs 100	Fare Rs 10; balance Rs 90
2	Middle, 08:00, balance Rs 100	Fare Rs 24; balance Rs 76
3	Outer, 17:00, balance Rs 100	Fare Rs 36; balance Rs 64
4	Outer, 10:00, balance Rs 100	Fare Rs 30; balance Rs 70
5	Outer, 20:00, balance Rs 100	Fare Rs 30; balance Rs 70
6	Inner, balance Rs 60	Accepted; remaining Rs 50
7	Inner, balance Rs 59.99	Denied; balance unchanged
8	Unknown zone	Rejected by validation
9	Negative recharge	Rejected by validation
10	Letters for hour	Rejected and re-prompted

Five-journey hand calculation: Inner off-peak Rs 10 + Middle peak Rs 24 + Outer off-peak Rs 30 + Inner peak Rs 12 + Middle off-peak Rs 20 = Rs 96. With an initial Rs 200 balance, the final balance is Rs 104 and the Rs 50 minimum is preserved.

A table-driven test loop takes O(t) time for t tests and O(t) storage for test cases. It is easier to add boundary cases than writing separate checks. Actual observed results and screenshots must be captured from the programs executed by the student.

Task 10 – Compare Expectations with Execution

For comparison, assume every journey costs Rs 30 and has no surcharge. Expected totals:

Journeys	Total rupees	Total paise
20,000	600,000	60,000,000
500,000	15,000,000	1,500,000,000
3,000,000	90,000,000	9,000,000,000

The 3,000,000-journey total of 9,000,000,000 paise exceeds the usual INT_MAX of 2,147,483,647, so int is unsafe for the paise total. long long holds these totals exactly. double and integer-paise totals may differ by tiny rounding error depending on how rupee fractions are represented and accumulated; the exact measured difference depends on the experiment and must be recorded from the actual run.

Typical sizes are int 4 bytes, long long 8 bytes and double 8 bytes; additions are O(1) for all. int may overflow, double may lose exactness for fractions, and long long paise is recommended. Compile using g++ -std=c++17 -Wall -Wextra source.cpp -o program. Warnings depend on the source and compiler; record actual warnings rather than inventing them.

Task 11 – Recommend and Defend the Final Design

Use SmartCard, Journey, FareTable and Passenger classes. Store money as long long paise, use enum class for zones, constexpr for fixed policy constants and explicit validation for zones, hours and recharges. Keep the balance private and deduct only after canTravel succeeds.

Fare changes should be made in one fare table or policy module. The original three-zone design is simple and takes O(1) time per journey. Its limitations are compile-time fare updates, unspecified concession percentages and increased complexity as rules grow. Processing n journeys is O(n).

A configuration file read at startup adds O(z) startup time and O(z) storage for z zone fares, but fare revisions need not require recompilation. Validate the configuration and version it. Run representative tests and record compiler version and elapsed time before making performance claims.

Task 12 – Re-evaluate and Adapt the Design Challenge

A. Impact analysis

Airport requires a fourth zone and Rs 60 fare (6000 paise). Update the zone type, fare table, validation, tests and documentation. Add a 10% weekend discount that replaces—not combines with—the peak surcharge. Add a MonthlyPass object storing validity and the two chosen zones.

B. Evaluate the original design

The switch can be extended with another case, but maintenance grows with each zone. A validated fare-table lookup scales more cleanly. SmartCard should continue managing balance; fare policy and monthly-pass eligibility should be separate responsibilities.

Worked examples: Airport on a weekday peak trip costs Rs 60 × 1.20 = Rs 72. Airport on a weekend costs Rs 60 × 0.90 = Rs 54, even when the time is normally peak, because the weekend discount replaces the peak surcharge.

C. Revised design code

```
enum class Zone { Inner, Middle, Outer, Airport };

constexpr long long FARES[] = {1000, 2000, 3000, 6000};

long long calculateFare(Zone z, bool peak, bool weekend) {

  int i = static_cast<int>(z);

  if (i < 0 || i >= 4) return -1;

  long long base = FARES[i];

  if (weekend) return base*90/100;

  if (peak) return base*120/100;

  return base;

}

class MonthlyPass { bool active; Zone first, second;

 public: MonthlyPass(bool a, Zone x, Zone y):active(a),first(x),second(y) {}

 bool covers(Zone z) const { return active && (z==first || z==second); }

};
```

Expected examples are 7200 paise (Rs 72) and 5400 paise (Rs 54). In a full program, format returned integer paise as rupees and paise. A pass-covered journey should not charge a single-journey fare, subject to pass validity and the minimum-balance policy. The assignment does not specify whether the Rs 50 requirement applies to pass journeys, so this must be confirmed.

D. Evidence-based defence

Decision: MODIFY the original design. Retain SmartCard encapsulation and integer paise, replace the switch with a centralized four-zone table, and add separate fare-policy and monthly-pass components. Both switch and array lookup are O(1); the array uses O(z) space and is easier to extend. Test weekday peak, weekday off-peak, weekend peak-time, Airport, invalid zone, covered zone and non-covered zone.

Conclusion and Submission Checklist

- Create GitHub repository DSA01-192511007-Assignment1.
- Add each task in its own folder with source code, input files and output screenshots.
- Add README.md with compiler/version, compile command, run steps and results.
- Commit progressively, at least one commit per task.

- Insert the working GitHub link on the cover page.
- Replace expected results with actual execution evidence and verify code on your compiler.